Deep Learning (DL)

Introduction to Perceptron, Gradient Descent, Forward Propagation, and Backward Propagation

IEOR 4578: Forecasting: *A Real-World Application*

Syed Waseem Haider, Ph.D.

Dr.Waseem.Haider@gmail.com

April 15, 2026

Introduction

- Reference Texts include:
  - Deep Learning by Ian Goodfellow, Yoshua Bengio, and Aaron Courville. Online version is available by authors here
  - Neural Networks and Deep Learning: A Textbook, 2019, by Charu C. Aggarwal.
- The flow of learning is as follows:
  - Understand the simple case of perceptron learning.
  - Building a binary Sign discriminant function.
  - Understanding the reasoning and mechanics behind the gradient descent algorithm.
  - Transitioning from discontinuous to a continuous activation function.
  - Understanding the backpropagation and deriving gradient for each architecture of Neural Network.
  - Creating efficiencies to train Neural Network models.

Perceptron

An Example of Decision Surface

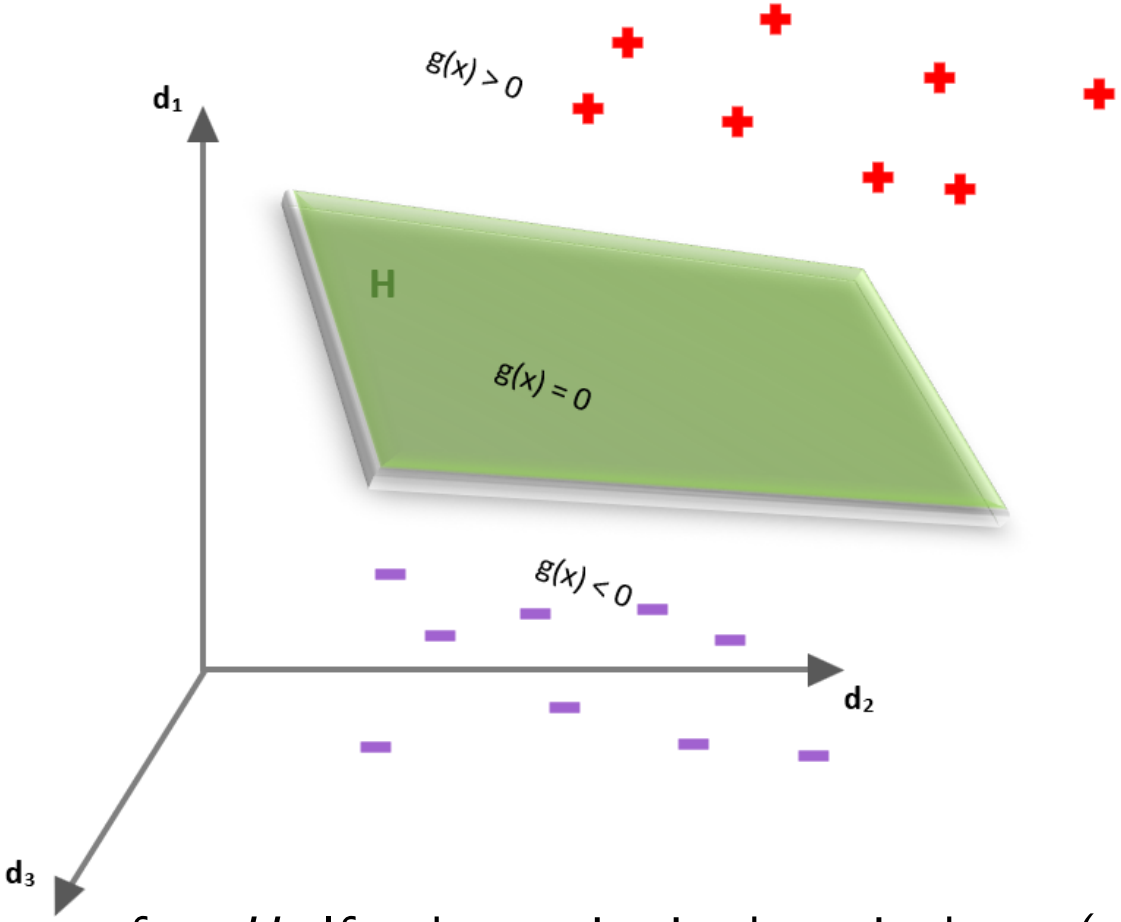

Figure: A decision surface H. If a data point is above it then $g(\mathbf{x}) > 0$, below it is $g(\mathbf{x}) < 0$, and on it is $g(\mathbf{x}) = 0$. Hence the decision surface $H = \{\mathbf{x} \mid g(\mathbf{x}) = 0\}$.

Perceptron

A Single Node Neural Network

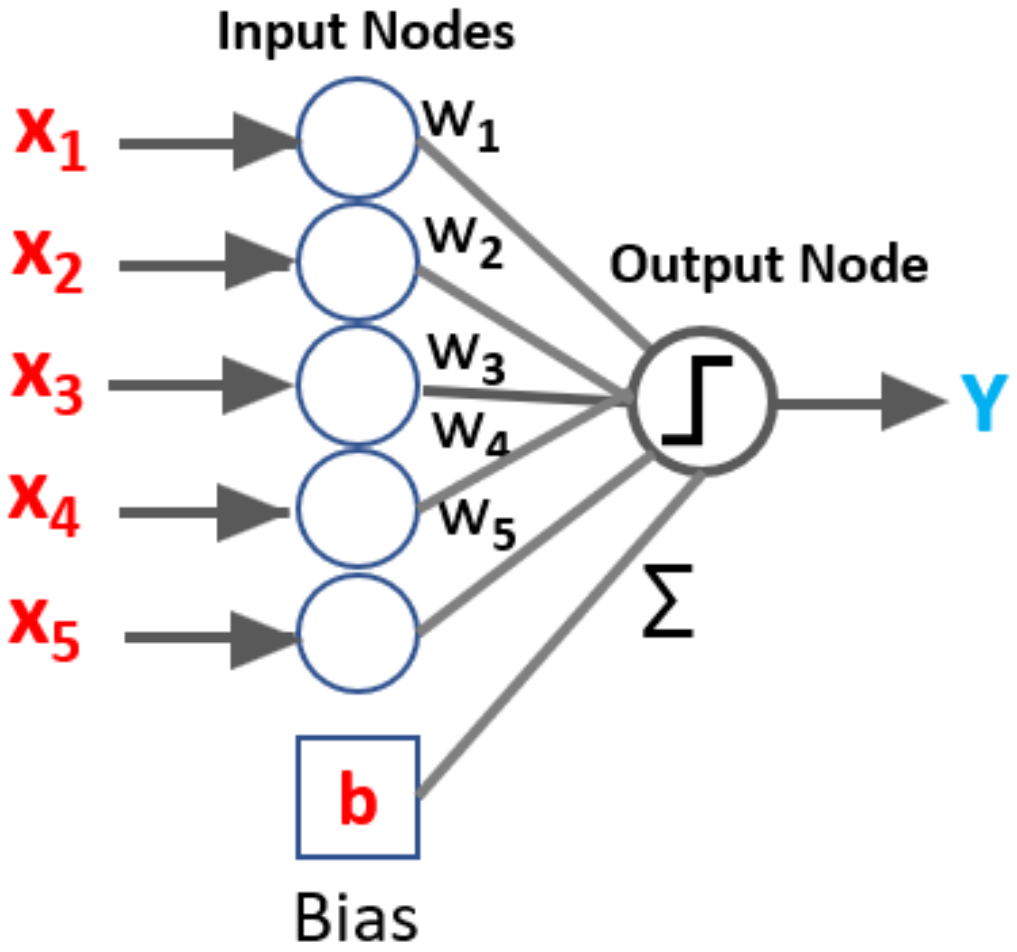

Figure: The decision surface can be modeled as a single node Neural Network is called the Perceptron (the model) $g(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$.

Perceptron

An Example of Decision Surface

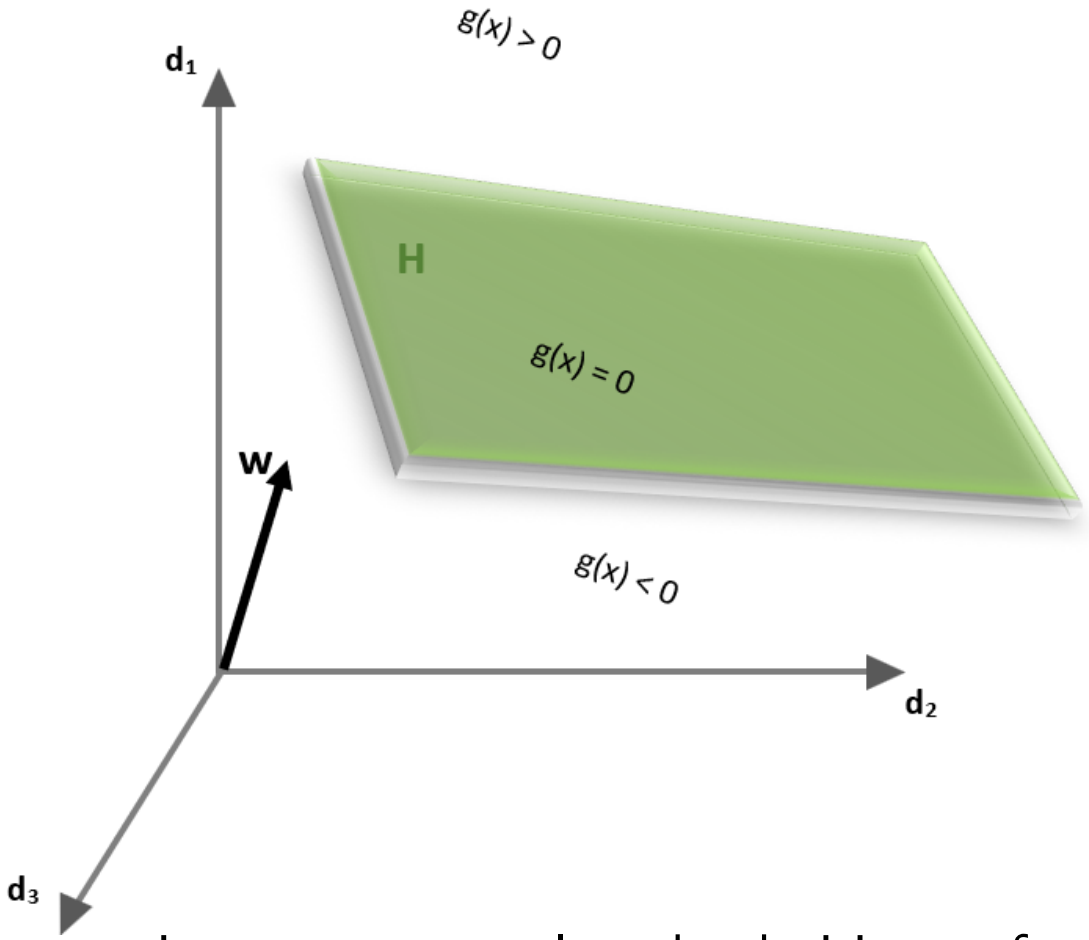

Figure: **w** is a vector normal to the decision surface, $\mathbf{w} \perp H$.

Perceptron

An Example of Decision Surface

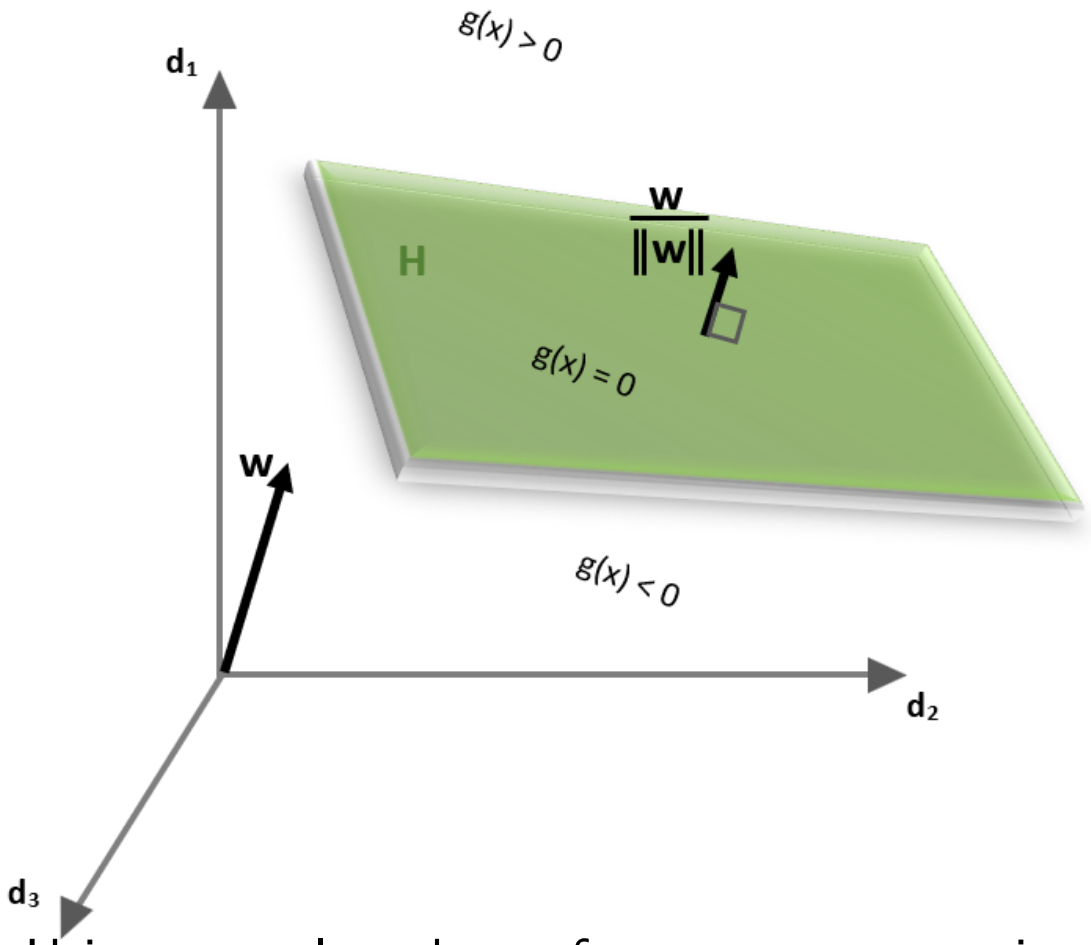

Figure: Using normal to the surface vector **w**, a unit vector is $\frac{\mathbf{w}}{||\mathbf{w}||}$.

Perceptron

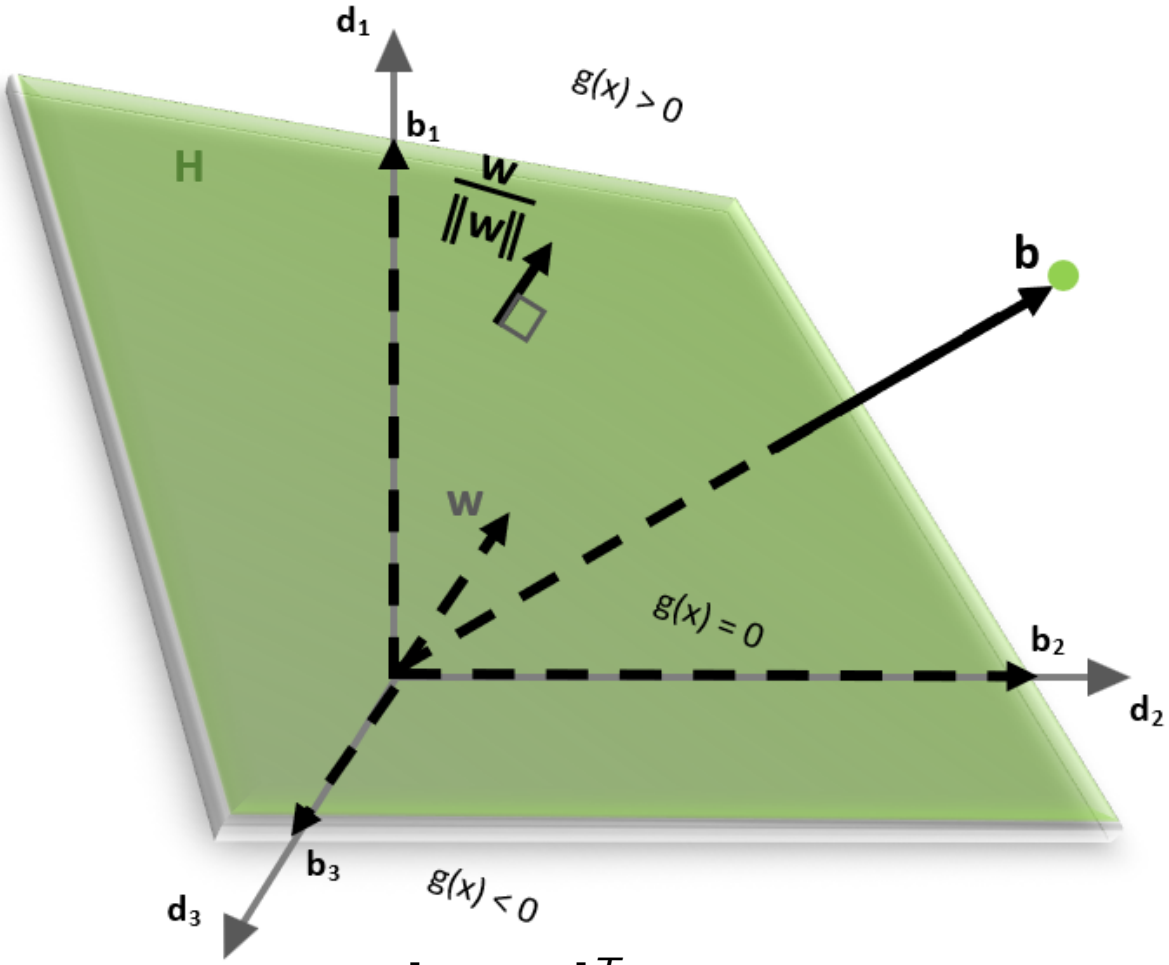

Figure: The intercept vector $\mathbf{b} = [b_1 b_2 b_3]^T$ is to the point lying on the decision surface H, such that the magnitude $b = ||\mathbf{b}||$ is the euclidean distance of the decision surface from the origin. If $b > 0$, the origin is on the positive side of the decision surface.

Perceptron

An Example of Decision Surface

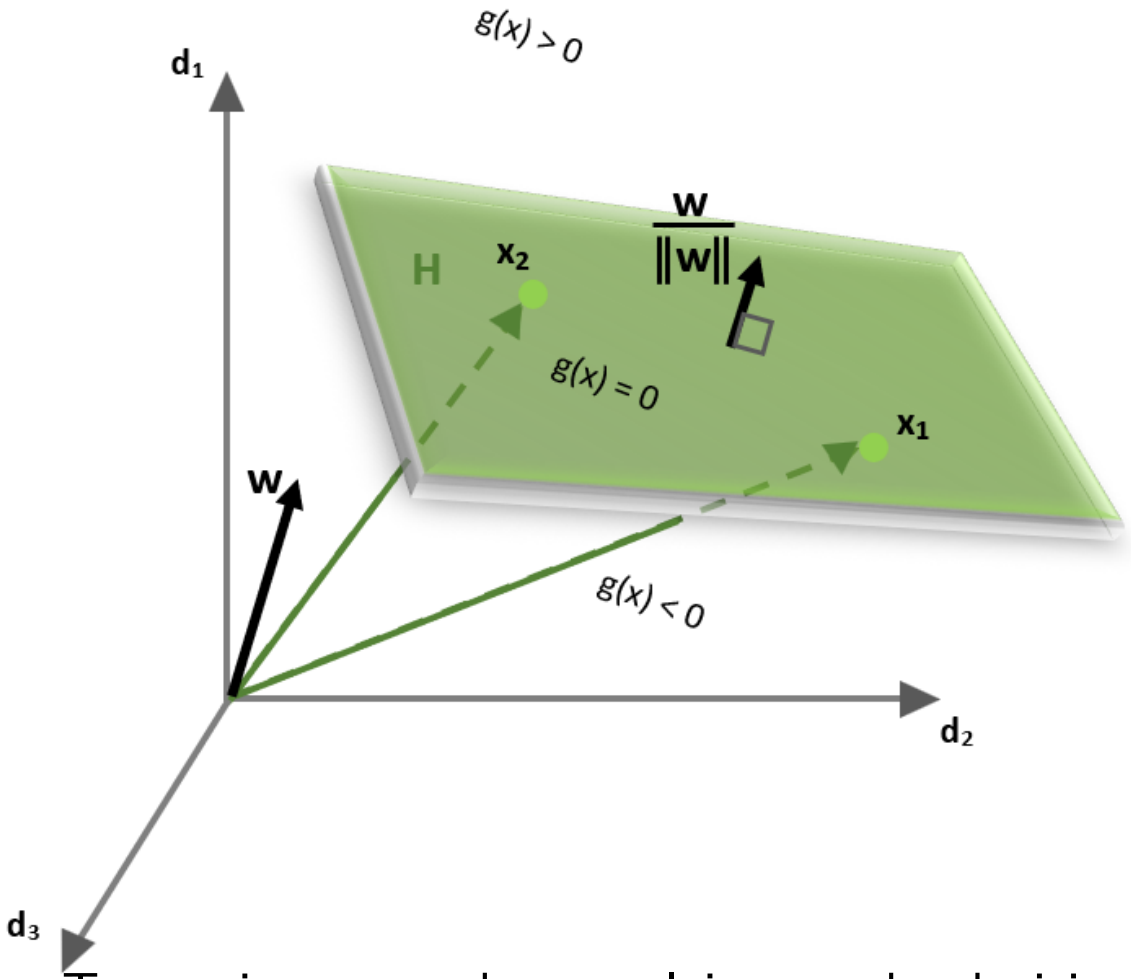

Figure: Two points $\mathbf{x}_1$ and $\mathbf{x}_2$ are lying on the decision surface H.

Perceptron

An Example of Decision Surface

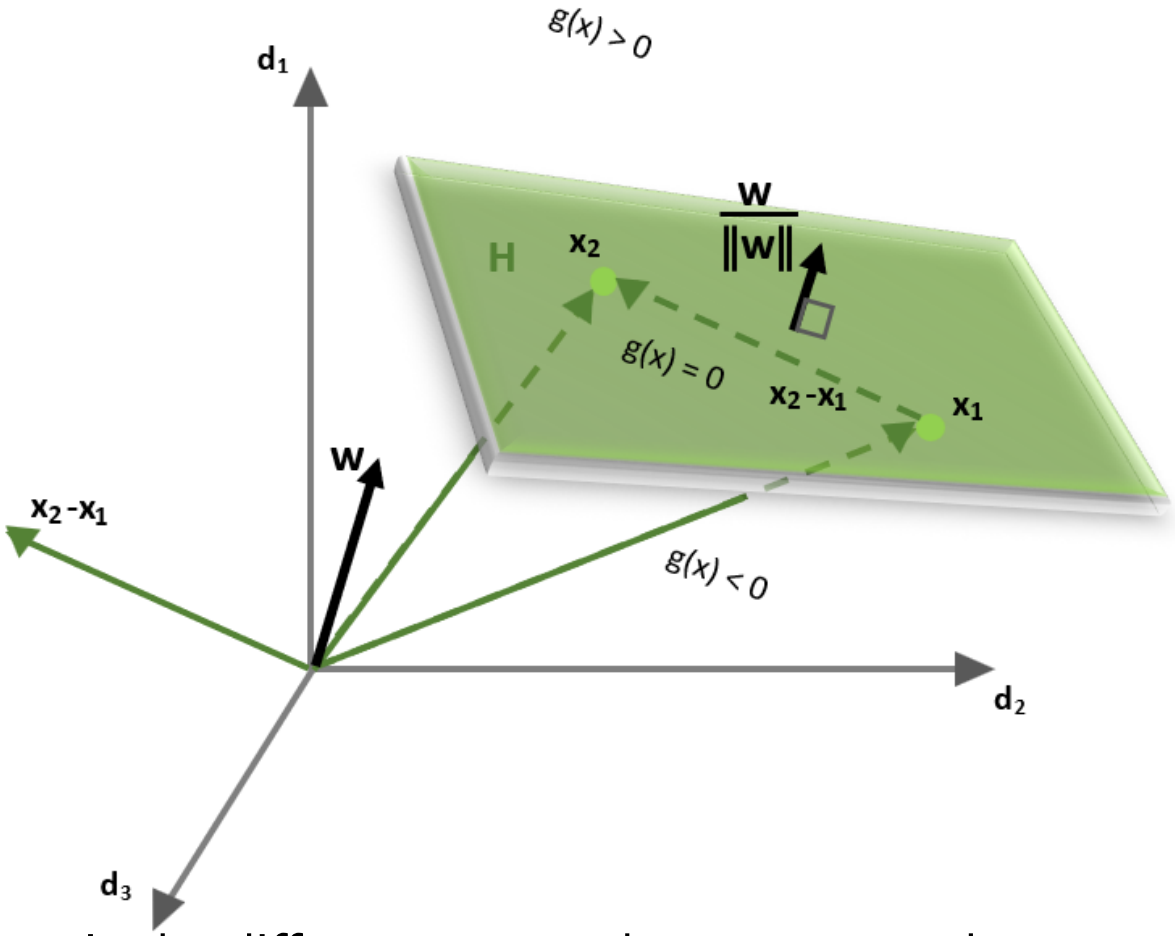

Figure: $\mathbf{x}_2 - \mathbf{x}_1$ is the difference vector between $\mathbf{x}_1$ and $\mathbf{x}_2$ vectors and lies on the decision surface, $\mathbf{x}_2 - \mathbf{x}_1 \parallel H$.

Perceptron

An Example of Decision Surface

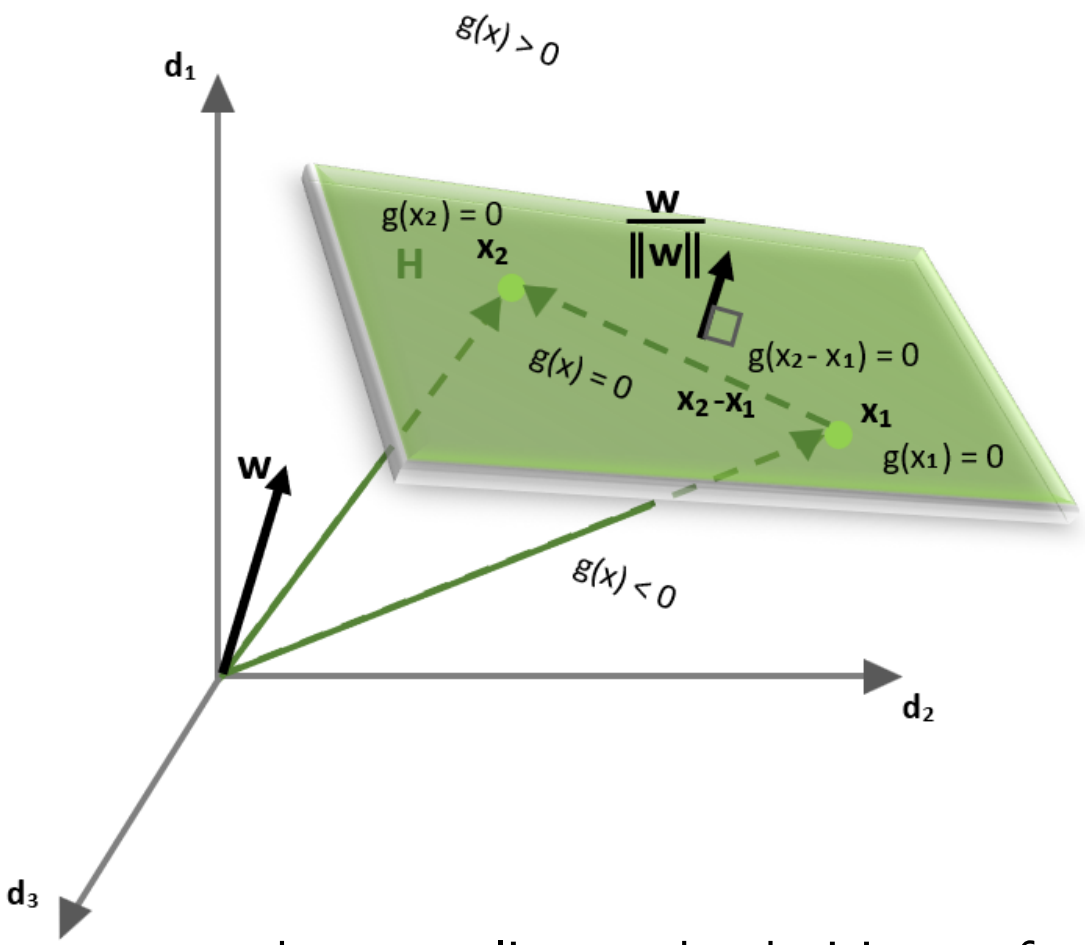

Figure: Vectors $\mathbf{x}_1$, $\mathbf{x}_2$, and $\mathbf{x}_2 - \mathbf{x}_1$ lies on the decision surface therefore $g(\mathbf{x}_1) = 0$, $g(\mathbf{x}_2) = 0$, and $g(\mathbf{x}_2 - \mathbf{x}_1) = 0$.

Perceptron

An Example of Decision Surface

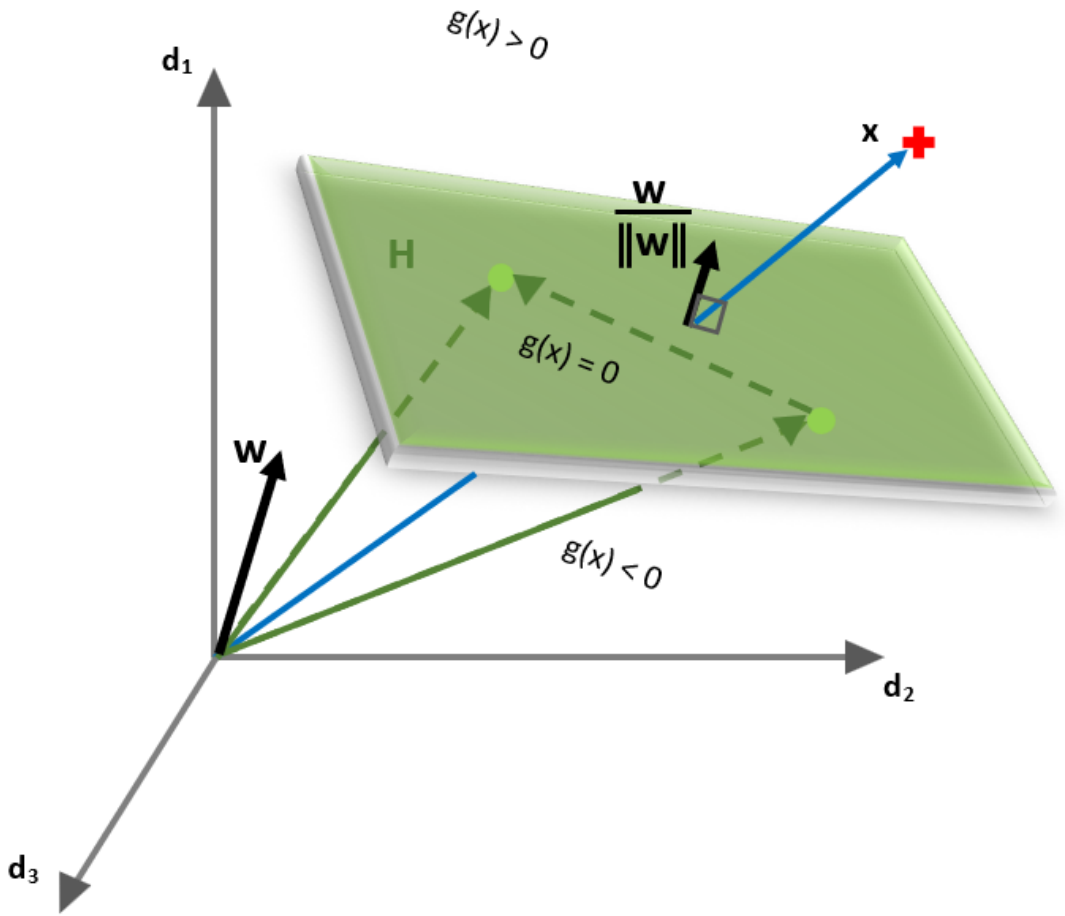

Figure: A point $\mathbf{x}$ lies on the positive side of the decision surface, $g(\mathbf{x}) > 0$. Now that $w, \mathbf{x}$, and b are defined, the discriminant function is $g(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$.

Perceptron

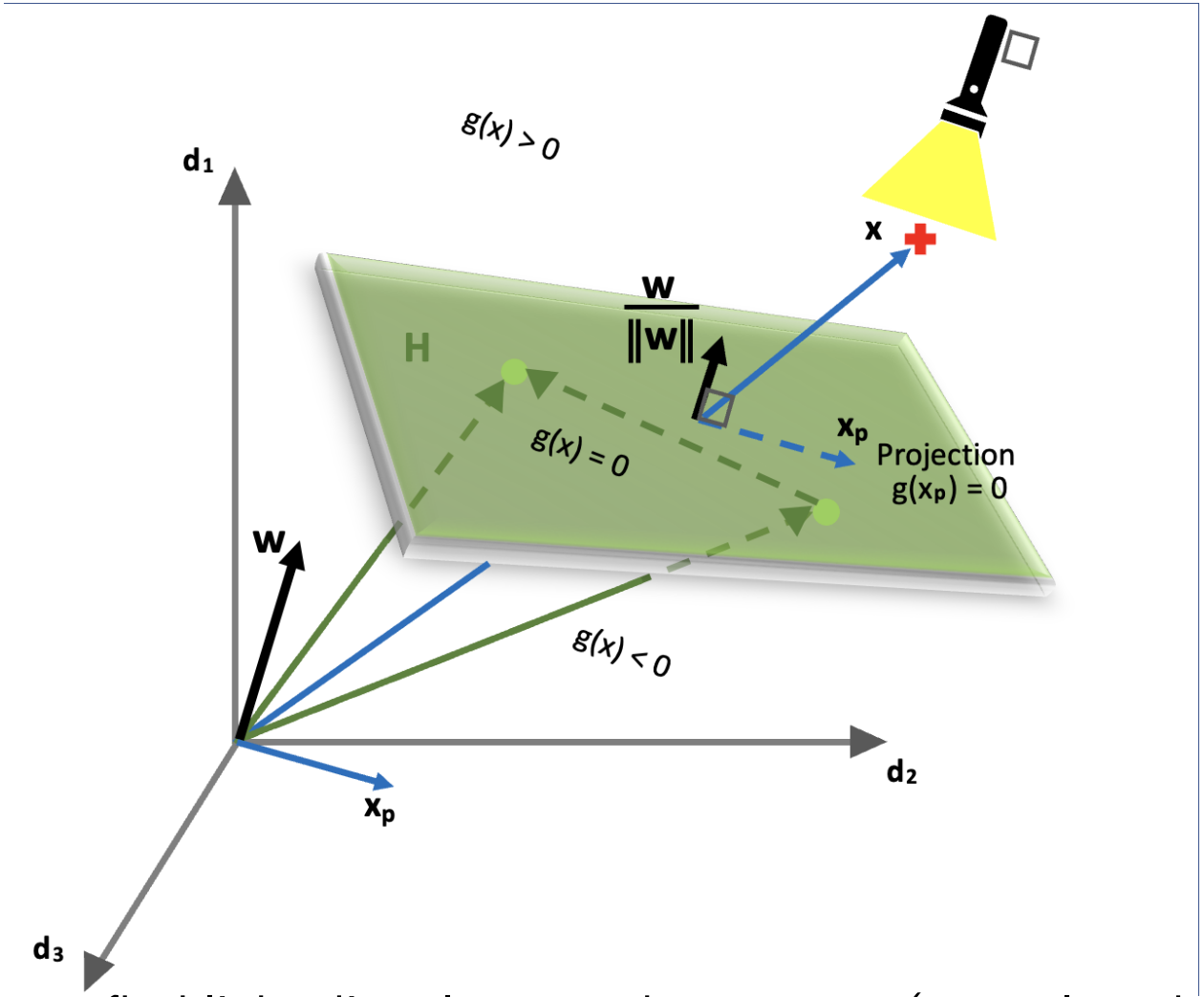

Figure: Shining a flashlight directly upon the vector $\mathbf{x}$ (normal to the $\mathbf{x}$) such that it casts a shadow, $\mathbf{x}_p$, on to the decision surface. $\mathbf{x}_p \parallel H$. $g(\mathbf{x}_p) = \mathbf{w}^T\mathbf{x}_p + b = 0$ $\Rightarrow \mathbf{w}^T\mathbf{x}_p = -b$.

Perceptron

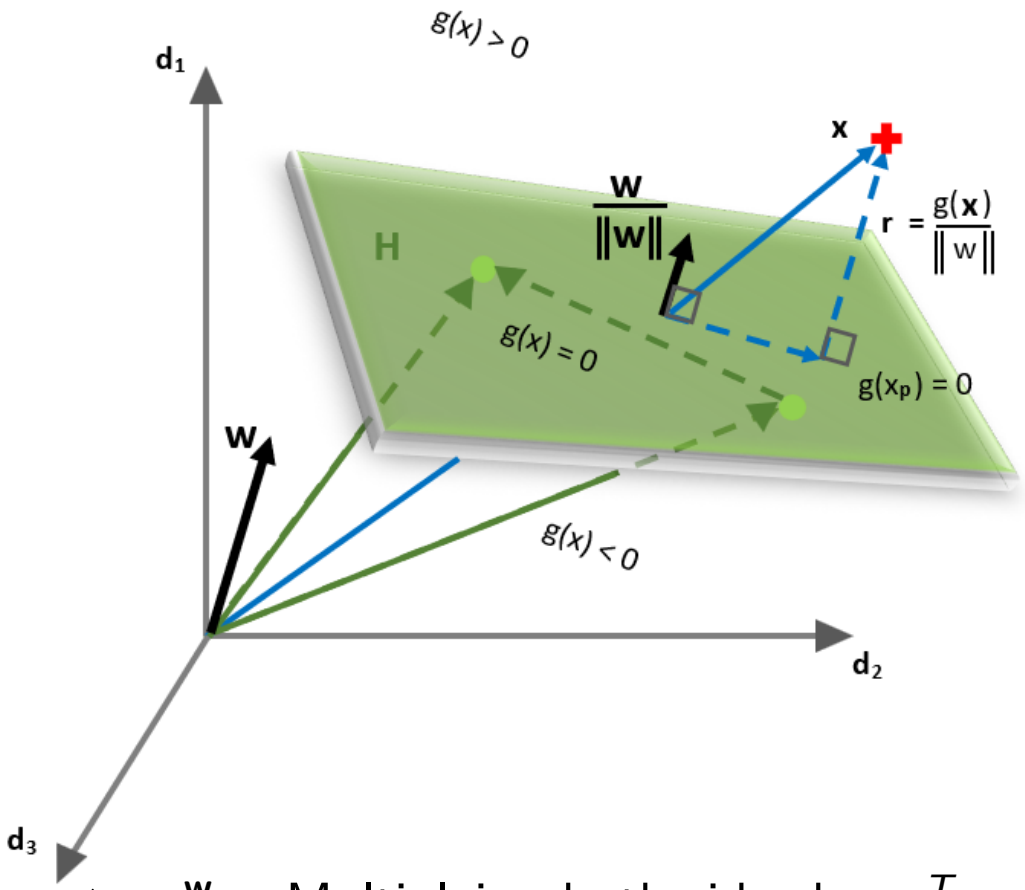

Figure: Let $\mathbf{x} = \mathbf{x}_p + r\frac{\mathbf{w}}{||\mathbf{w}||}$. Multiplying both sides by $\mathbf{w}^T$, $\mathbf{w}^T\mathbf{x} = \mathbf{w}^T\mathbf{x}_p + r\frac{\mathbf{w}^T\mathbf{w}}{||\mathbf{w}||}$, where $\frac{\mathbf{w}^T\mathbf{w}}{||\mathbf{w}||} = \frac{||\mathbf{w}||^2}{||\mathbf{w}||}$, and $g(\mathbf{x}_p) = \mathbf{w}^T\mathbf{x}_p + b = 0 \Rightarrow \mathbf{w}^T\mathbf{x} = -b + r||\mathbf{w}||$ $\Rightarrow r = \frac{g(\mathbf{x})}{||\mathbf{w}||}$.

Perceptron

An Example of Decision Surface

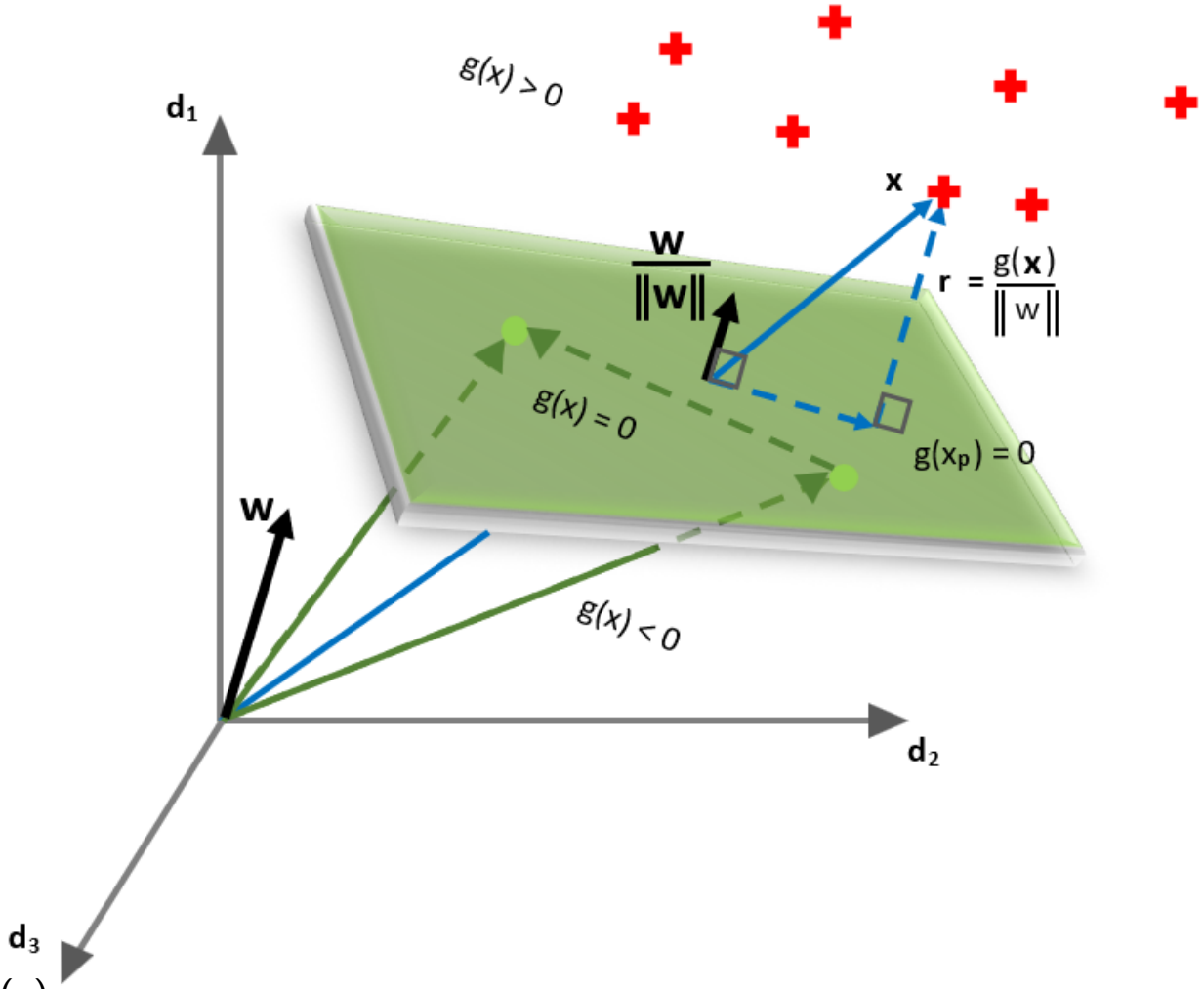

Figure: $r = \frac{g(\mathbf{x})}{||\mathbf{w}||}$ provides the distance from the decision surface H to any point $\mathbf{x}$.

Perceptron

An Example of Decision Surface

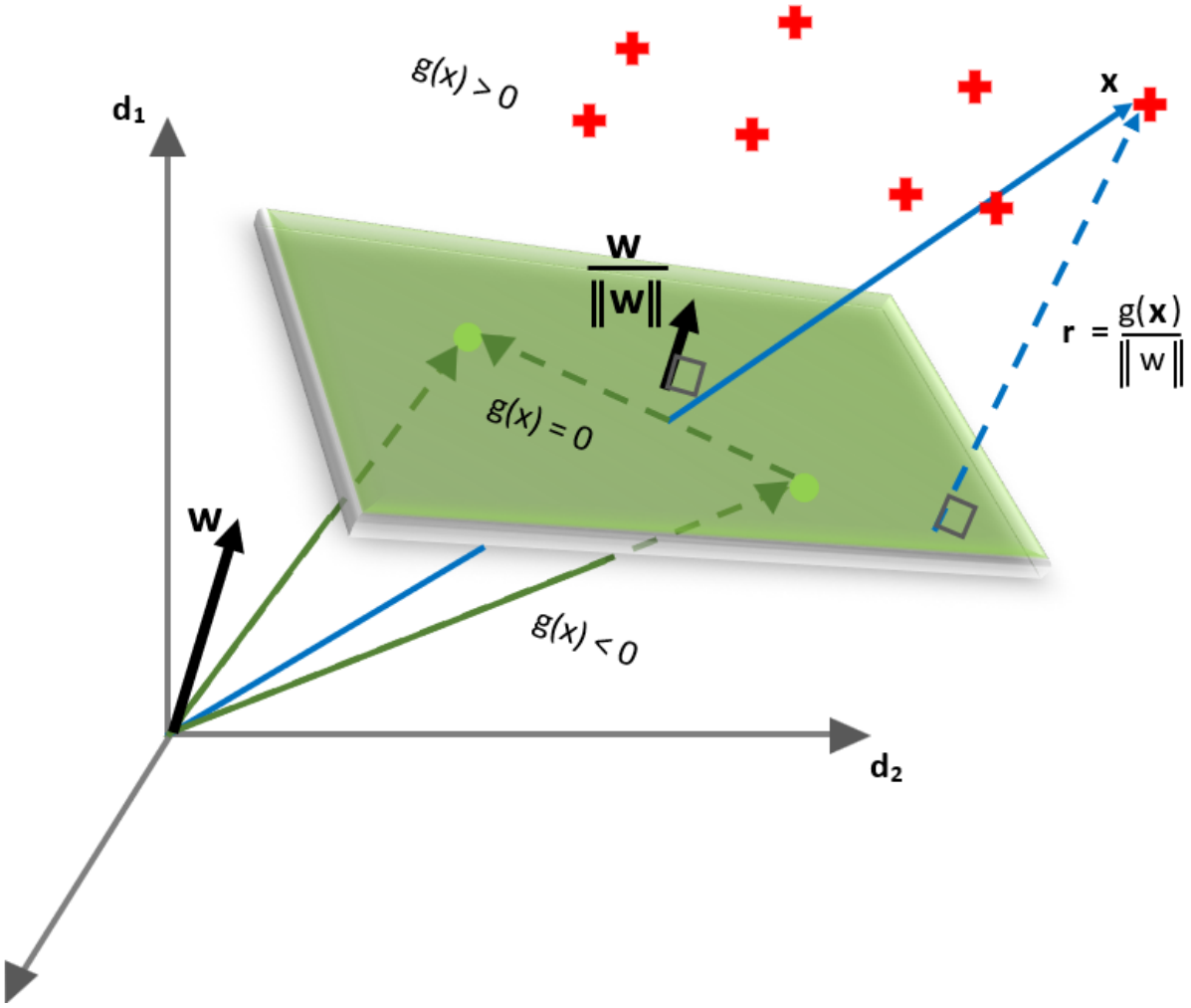

Figure: $r = \frac{g(\mathbf{x})}{||\mathbf{w}||}$ provides the distance from the decision surface H to any point $\mathbf{x}$.

Perceptron

An Example of Decision Surface

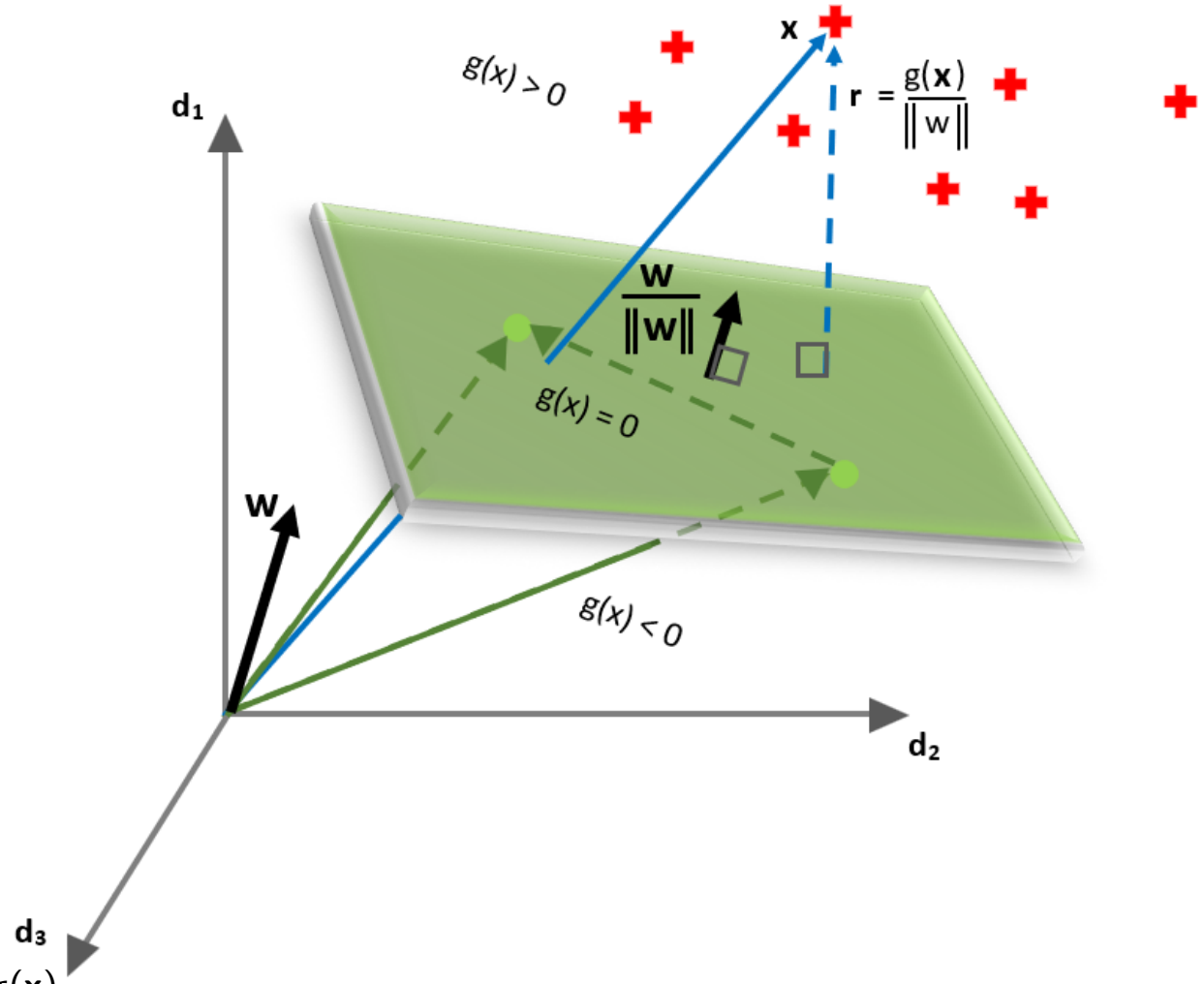

Figure: $r = \frac{g(\mathbf{x})}{||\mathbf{w}||}$ provides the distance from the decision surface H to any point $\mathbf{x}$.

Perceptron

An Example of Decision Surface

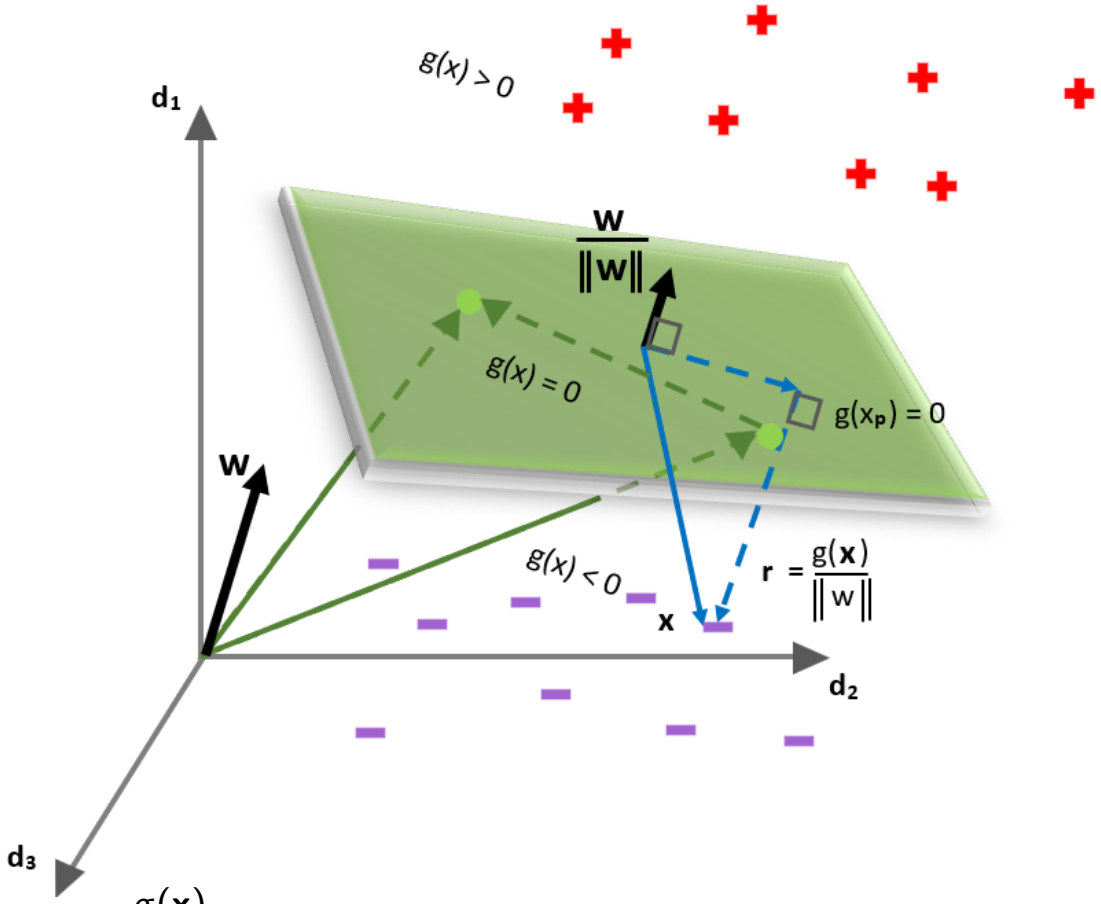

Figure: Similarly, $r = \frac{g(\mathbf{x})}{||\mathbf{w}||}$ provides the distance from the decision surface H to any point $\mathbf{x}$ on the either side. Either $g(\mathbf{x}) > 0$, or $g(\mathbf{x}) < 0$ depending if $\mathbf{x}$ is above or below H.

Perceptron

A Hyperplane as a Decision Surface in an n-Dimensional Space.

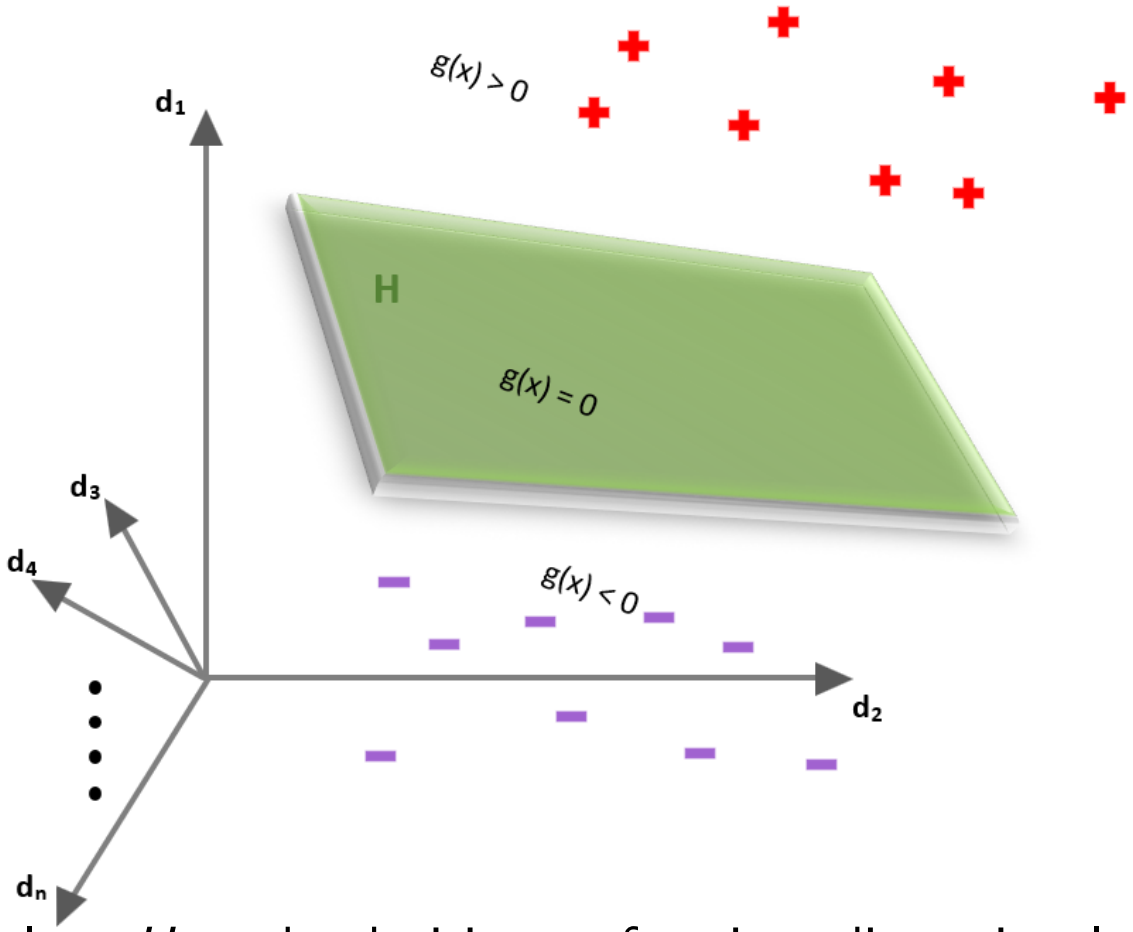

Figure: A hyperplane H as the decision surface in n-dimensional space with the discriminant function $g(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b = \sum_{i=1}^{n} w_i x_i + b$.

Perceptron

- $g(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b = \sum_{i=1}^{n} w_i x_i + b$ can be improved by incorporating the bias b into the summation, $g(\mathbf{x}) = \sum_{i=0}^{n} w_i x_i$.
- Intuitively, $g(\mathbf{x})$ provides the direction ($\pm$), and the scaled distance, $r||\mathbf{w}||$, from the point $\mathbf{x}$ to H.
- The direction ($\pm$) identifies if the point $\mathbf{x}$ lies above or below H.
- One may only be interested in the direction ($\pm$), and not necessarily in the distance of the observation. For example, a dichotomous classification such as True-False, Good-Bad, High-Low, Action-Ignore, etc.
- Such functions can be defined as $g(\mathbf{x}) = sign(\mathbf{w}^T\mathbf{x}) = sign(\sum_{i=0}^{n} w_i x_i)$.

Perceptron

Optimization – Objective Function

- An objective function aka cost function, aka loss function, and aka error function minimizes the error between the observations and the predicted or estimated outputs.
- The objective function could be the sum of the square of errors (to accommodate the direction of errors), $\sum_{i=1}^{N} (y_i - \hat{y}_i)^2$.
- The function being approximated may run into several different types of numerical errors such as poor conditioning where a small change in the input could blow up the output.
- Usually x is considered the input which it is. But for optimization, input is considered the weight or parameter w which is tunable.
- When changing w the error $y_i - \hat{y}_i$ will change. For optimal values of w, the error will be minimum.

Perceptron

Optimization – Gradient Descent

- Therefore, the change in the output for a change in the input is defined and studied as the gradient, $\Delta = \frac{dy}{dx} = \lim_{\Delta x \to 0} \frac{\Delta y}{\Delta x} = f'(x)$.
- At the rate Δ, an incremental input η will result in the output $\eta f'(x)$.
- Using the concept of gradient and incremental, the objective function is defined as $J(w + \eta) \approx J(w) + \eta J'(w)$.
- When trying to minimize the objective function $J(w)$, which represents the loss function, the size and the direction of the incremental η become important. $J(w - \eta) < J(w + \eta)$, or $J(w - \eta) > J(w + \eta)$.
- In other words, $J(w) - \eta J'(w) < J(w) + \eta J'(w)$, or $J(w) - \eta J'(w) > J(w) + \eta J'(w)$.

Perceptron

Optimization – Gradient Descent

- To address the direction in which $J(w)$ is reducing, $\mathbf{d}$ the direction is introduced such that $J(w + \eta\mathbf{d}) \approx J(w) + \eta J'(w)\mathbf{d}$.
- When the right direction is chosen, $J(w + \eta\mathbf{d}) < J(w)$.
- The discussion so far assumes that the gradient (or rate) $J'(w)$ is fixed, which is usually not the case.
- Consider the convex function $J(w)$ where the slope of the line $J'(w)$ is changing continuously and at one value of w it becomes zero.
- Minimizing or maximizing is finding that value of w where the gradient $J'(w) = 0$.
- This leads to defining $w_{k+1} = w_k + \eta J'(w)\mathbf{d}$ such that $J(w_{k+1}) < J(w_k)$.
- $w_{k+1} = w_k + \eta J'(w)\mathbf{d}$ is called the **gradient descent**, or the **steepest descent**.

Perceptron

Gradient Descent Algorithm

- The problem of minimizing the error function, which was reduced to finding the optimal weights, with the gradient descent, is now reduced to determining the step size η and the direction $\mathbf{d}$ with an initial w_0.
- Simple Gradient Descent Algorithm:

  begin: Initialize δ, $\eta(\cdot)$, $d(\cdot)$, $k \leftarrow 0$, $w(0)$:

  do:

  $k \leftarrow k + 1$

  $w(k+1) \leftarrow w(k) + \eta(k) J'(w(k)) d(k)$

  <u>until</u>: $|\eta(k) J'(w(k)) d(k)| < \delta$

  return: $w(k)$

  end:

- The acceptable threshold is defined in advance, while the number, series of steps, and directions are searched using the method in a zig-zag way.

Perceptron

Gradient Descent Algorithm

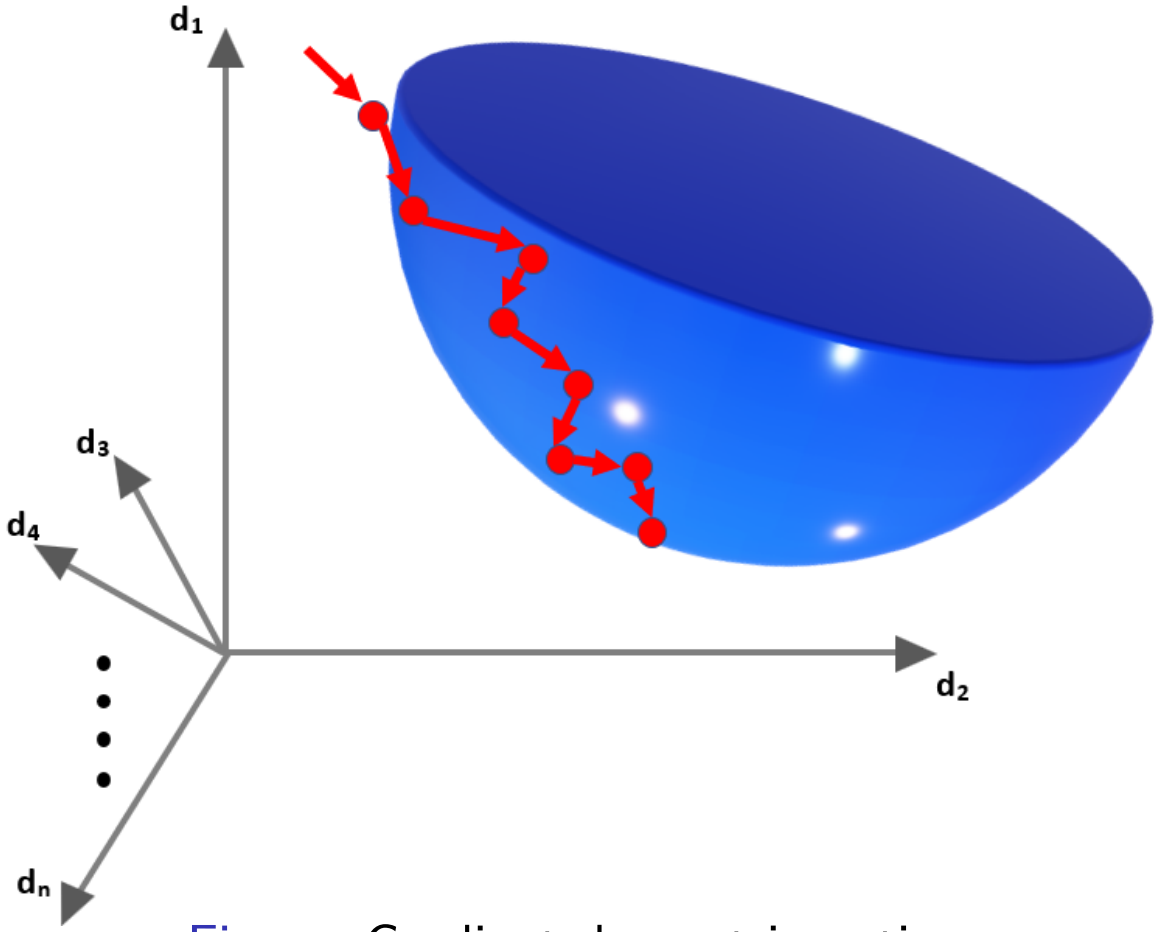

Figure: Gradient descent in action.

Perceptron

Gradient Descent Algorithm

- $J'(w(k))d(k)$ can be simplified if it's assumed the $J'(w(k))$ provides both the magnitude of the gradient and the direction, so when searching using $w(k+1) = w(k) + \eta(k)J'(w(k))$ the direction is implied.
- The first-order derivative under represents a convex shape function. Therefore, it cannot confirm that if $J'(w(k)) = 0$, k is a local optimal point.
- Finally, the size of step $\eta(k)$ can make learning difficult. For example, if it's too small, it will take longer to find the local minimum. Similarly, too big will make it overshoot causing again a slow search.
- This is overcome by following Newton's Descent method.
- However, first things first – some additional concepts to follow.

Perceptron

Gradient Descent Algorithm

- Hessian matrix, which organizes the second-order partial derivatives of a function of many variables, provides a way to measure the curvature of the function $J(w)$ along each dimension w.
- So far we have only studied variations along a single dimension, but it can be easily extended to p dimensions.
- Hessian matrix summarizes the curvatures from all dimensions into a single dimension or scalar form, $\mathbf{H} : \mathbb{R}^p \rightarrow \mathbb{R}$.
- A Hessian matrix is a gradient of a gradient, $\nabla\ (\nabla J)$.
- A Hessian matrix is positive semidefinite, i.e. its determinant, $det(\mathbf{H}) \geq 0$ i.e., a saddle or minimum point that some eigenvalues of $det(\mathbf{H})$ will be positive and others negative, or all will be positive.

Perceptron

Gradient Descent Algorithm

- Which is important because the $det(\mathbf{H}) = 0$ at $w(opt)$ and $det(\mathbf{H}) > 0$ in either direction, therefore the loss function f will be minimum.
- Hessian matrix $\mathbf{H}$ solves for if the gradient step $\eta(k)$ will improve in each direction sufficiently. Hence hinting towards a relationship between $\mathbf{H}$ and $\eta(k)$.
- For example, let $f(x_1) = 3x_1^2$, and $f(x_2) = 0.0003x_2^2$. First derivatives are $f'(x_1) = 6x_1$, and $f'(x_2) = 0.0006x_2$, respectively.
- $f(x_1)$ and $f(x_2)$ are minimum at $(0, 0)$.

Perceptron

Gradient Descent Algorithm

- The searches along $x_{1,k+1} = x_{1,k} + \eta 6 x_{1,k}$, and $x_{2,k+1} = x_{2,k} + \eta 0.0006 x_{2,k}$ demonstrate that the gradient step does not impact expectedly along each dimension to get from $(1,1) \rightarrow (0,0)$.
- However, when $f''(x_1) = 6$, and $f''(x_2) = 0.0006$ divide the gradients such that $x_{1,k+1} = x_{1,k} + \eta f''(x_1)^{-1} 6 x_{1,k} \Rightarrow x_{1,k+1} = x_{1,k} + \eta x_{1,k}$, and $x_{2,k+1} = x_{2,k} + \eta f''(x_2)^{-1} 0.0006 x_{2,k} \Rightarrow x_{2,k+1} = x_{2,k} + \eta x_{2,k}$, the gradient step converges similarly in each dimension resulting in a faster convergence potentially.
- Taylor series expansion allows a function around a point a to be expressed in a series of increasing orders of derivatives.
- For example, $J(w) = J(a) + J'(a)(w-a) + \frac{J''(a)}{2!}(w-a)^2 + \frac{J'''(a)}{3!}(w-a)^3 + \ldots$.

Perceptron

Gradient Descent Algorithm

- For the gradient descent algorithm, Taylor series expansion up to the second derivative around the point w_{opt} is of interest.
- At $w(opt)$, $J(w)$ will reach the curvature's peak point where $J'(w(opt)) = 0$, and $J''(w(opt)) > 0$ implying the local minimum.
- Expanding Taylor series around $w(k)$ up to the second derivative, $J(w) \approx J(w(k)) + J'(w(k))(w - w(k)) + \frac{J''(w(k))}{2!}(w - w(k))^2$.
- Writing in terms of vectors and Hessian matrix, $J(\mathbf{w}_{k+1}) \approx J(\mathbf{w}_k) + \nabla J\,(\mathbf{w}_{k+1} - \mathbf{w}_k) + \frac{1}{2}(\mathbf{w}_{k+1} - \mathbf{w}_k)^T \mathbf{H}(\mathbf{w}_{k+1} - \mathbf{w}_k)$.
- In the above, substituting $\mathbf{w}(k+1) = \mathbf{w}(k) - \eta(k)\nabla J$ leads to $J(\mathbf{w}_{k+1}) \approx J(\mathbf{w}_k) - \eta_k ||\nabla J||^2 + \frac{1}{2}\eta_k^2 \nabla J^T \mathbf{H} \nabla J$.

Perceptron

Gradient Descent Algorithm

- Using the fact that at the local minimum $J(\mathbf{w}_{k+1}) = J(\mathbf{w}_k)$, and the changes in η_k must be 0, $\frac{d}{d\eta_k}\ \eta_k = 0$.
- $-\eta_k||\nabla J||^2 + \frac{1}{2}\eta_k^2 \nabla J^T \mathbf{H} \nabla J = 0$, when $J(\mathbf{w}_{k+1}) = J(\mathbf{w}_k)$.
- $\frac{d}{d\eta_k}\left[-\eta_k||\nabla J||^2 + \frac{1}{2}\eta_k^2 \nabla J^T \mathbf{H} \nabla J\right] = 0$.
- $-||\nabla J||^2 + \eta_k \nabla J^T \mathbf{H} \nabla J = 0$.
- $||\nabla J||^2 = \eta_k \nabla J^T \mathbf{H} \nabla J \implies \eta_k = \frac{||\nabla J||^2}{\nabla J^T \mathbf{H} \nabla J}$, where η_k is a constant independent of k, and $\mathbf{H}$ is a constant.
- In $\mathbf{w}(k+1) = \mathbf{w}(k) - \eta(k)\nabla J$ substituting back $\eta_k = \frac{||\nabla J||^2}{\nabla J^T \mathbf{H} \nabla J}$ results in $\mathbf{w}_{k+1} = \mathbf{w}_k - \frac{||\nabla J||^2}{\nabla J^T \mathbf{H} \nabla J}\nabla J = \mathbf{w}_k - \frac{\nabla J^T \nabla J}{\nabla J^T \mathbf{H} \nabla J}\nabla J$.
- $\mathbf{w}_{k+1} = \mathbf{w}_k - \mathbf{H}^{-1}\nabla J$.
- Note: In many texts $J'(w(k))$ is written as ∇J to pronounce gradient.

Perceptron

Newton Descent Algorithm

The Newton Descent Algorithm:

begin: Initialize threshold δ, and $w(0)$:

do:

$k \leftarrow k + 1$

$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \mathbf{H}^{-1}\nabla J$

<u>until</u>: $|\mathbf{H}^{-1}\nabla J| < \delta$

return: $w(k)$

end:

Perceptron

- The discriminant function (the model) $g(\mathbf{x}) = sign(\mathbf{w}^T\mathbf{x}) = sign(\sum_{i=0}^{n} w_i x_i)$ is a discontinuous function getting values from $\{-1, 1\}$.
- Using Mean Square Errors (MSE), the objective function for i-th training point, $J_i(w) = \frac{1}{2}\left(y_i - sign(\mathbf{w}^T\mathbf{x}_i)\right)^2 = 1 - y_i\ sign(\mathbf{w}^T\mathbf{x}_i)$ is also a discontinuous function getting values from $\{-2, 0, 2\}$.
- The 1 in above is because y_i and $sign(\mathbf{w}^T\mathbf{x}_i)$ are in $\{-1, 1\}$, therefore $\frac{1}{2}\left(y_i^2 + sign^2(\mathbf{w}^T\mathbf{x}_i)\right) = 1$.
- The objective function cannot be readily used in the Newton Gradient Descent algorithm because it's a discontinuous function.
- The first-order derivative J' is either not defined on discontinuities, or is '0' because the function takes on the constant values.

Rectified Linear Unit (ReLU) Activation

- One way to overcome this is to use a Rectified Linear Unit (ReLU) method, $J_i(w(k)) = \text{MAX}\ \{-y_i(\mathbf{w}(k)^T\mathbf{x}_i), 0\}$, which maps to $[0, \infty)$.
- $J_i(w(k)) > 0$ the error grows linearly based on the distance of the misclassification.
- $|\mathbf{H}^{-1}J_i'(w_j(k))|$ will reduce the smaller is $J_i'(w_j(k)) = \text{MAX}\ \{-y_i\ w_j(k)x_{j,i}, 0\}$.
- Despite the Linear term in the name of ReLU, it's a non-linear activation function, because it does not conform to the standard linear equation $y = mx + c$. It is only piece-wise linear.
- Linear regression problems are not a good choice for Gradient Descent one it's slow and the other Hessian matrix will be '0' for the equation linear in parameters.
- The discrimnant function $g(\mathbf{x})$ can be generalized as an activation function form $\hat{y} = \Phi(\mathbf{w}^T\mathbf{x})$, or simply $\Phi(v)$.

Softmax Activation

Multiclass Prediction – Multinomial Distribution

- A multiclass prediction problem scales upon the binary class prediction problem discussed earlier. The idea is to predict the class correctly among several possible classes.
- Softmax activation function $\Phi(v)_i = \frac{exp(v_i)}{\sum_{j=1}^{k} exp(v_j)} \quad \forall i \epsilon \{1, \ldots, k\}$, where $\Phi(\cdot)_i \ \epsilon \ \{0, 1\}$.
- $v_i = \mathbf{w}^T \mathbf{x} + b$ is the hyperplane for separating the i-th class from the rest of the classes, where for the i-the class $v_i > 0$, and < 0 for the other classes.
- Intuitively, $\Phi(v)_i = 1$ if it's i-th class. Conversely, $\Phi(v)_j = 0, \forall j \neq i$.

Softmax Activation

Multiclass Prediction – Multinomial Distribution

- If MAX $\{P[y=1|x], \ldots, P[y=k|x]\} = P[y=i|x]$ $\Rightarrow \Phi(v)_i = 1 \Rightarrow \hat{y} = i$ is the prediction.
- Because the multinomial distribution belongs to the EDM family, the objective function is defined as MLE instead of MSE.
- Therefore, the loglikelihood fucntion of Softmax is $log\ \Phi(v)_i = v_i - log \sum_{j=1}^{k} exp(v_j)$.
- For a correct prediction, MLE is $log\ \Phi(v)_i \approx 0$.
- The term soft comes from the function being continuous and differentiable.

Sigmoid Activation

Binaryclass Prediction – Bernoulli Distribution

- Simplifying choices also simplify model, $\Phi(v)_i = \frac{exp(v_i)}{\sum_{j=1}^{2} exp(v_j)} \quad \forall i \epsilon \{1, 2\}$, where $\Phi(\cdot) \ \epsilon \ [0, 1]$.
- It is called the Sigmoid activation function $\epsilon [0, 1]$, where $\hat{y} \epsilon \{1, 2\}$.
- Similar to Softmax, for a correct prediction, MLE is $log \ \Phi(v)_i \approx 0$.

Back Propagation

Scaling from a Single Perceptron to a Multilayer Neural Network

- Let $w \to x \to y \to z$ be a dependency graph such that a change in w propagates through a change in z.
- For recursive functions $z = f(g(h(w)))$, where $z = f(y)$, $y = g(x)$, and $x = h(w)$, the chain rule of differentiation is $\frac{\partial z}{\partial w} = \frac{\partial z}{\partial y}\frac{\partial y}{\partial x}\frac{\partial x}{\partial w}$.

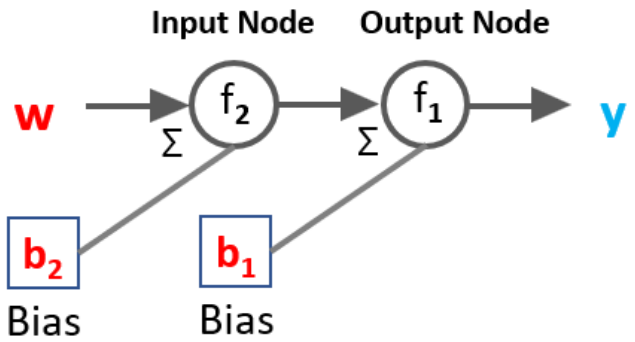

Figure: A sequence of nodes maps input w to output $y = f_1(f_2(w + b_2) + b_1)$ recursively.

- The gradient from input to output (or output to input) is $\frac{\partial f_1(f_2(w+b_2)+b_1)}{\partial w}$.
- Using chain rule expanding the gradient $\frac{\partial f_1(f_2(w+b_2)+b_1)}{\partial w} = \frac{\partial f_1(f_2(w+b_2)+b_1)}{\partial f_2(w+b_2)} \frac{\partial f_2(w+b_2)}{\partial w}$.

Back Propagation

Scaling from a Single Perceptron to a Multilayer Neural Network

Figure: Extending the sequence of nodes to k nodes and ignoring the bias to simplify analysis, $y = f_1(f_2(\ldots(f_{k-1}(f_k(w))\ldots))$ recursively.

- The gradient from input to output is $\frac{\partial f_1(f_2(\ldots(f_{k-1}(f_k(w))\ldots)))}{\partial w}$.
- Again, using chain rule expanding on the changes from input to output is
$$\frac{\partial f_1(f_2(\ldots(f_{k-1}(f_k(w))\ldots)))}{\partial f_2(\ldots(f_{k-1}(f_k(w))\ldots)} \frac{\partial f_2(\ldots(f_{k-1}(f_k(w))\ldots)}{\partial f_3(\ldots(f_{k-1}(f_k(w))\ldots)} \cdots \frac{\partial f_{k-1}(f_k(w))}{\partial f_k(w)} \frac{\partial f_k(w)}{\partial w}.$$
- Using $\frac{\partial f_{i-1}(\ldots(f_{k-1}(f_k(w))\ldots)}{\partial f_i(\ldots(f_{k-1}(f_k(w))\ldots)} = \frac{\partial f_{i-1}(f_i)}{\partial f_i(f_{i+1})}$, summarising the chain rule $\frac{\partial f_1(f_2(\ldots(f_{k-1}(f_k(w))\ldots)))}{\partial w} = \prod_{i=2}^{k+1} \frac{\partial f_{i-1}(f_i)}{\partial f_i(f_{l+1})}$, where at $i = k+1, f_{k+1}(w) = w$.

Back Propagation

Scaling from a Single Perceptron to a Multilayer Neural Network

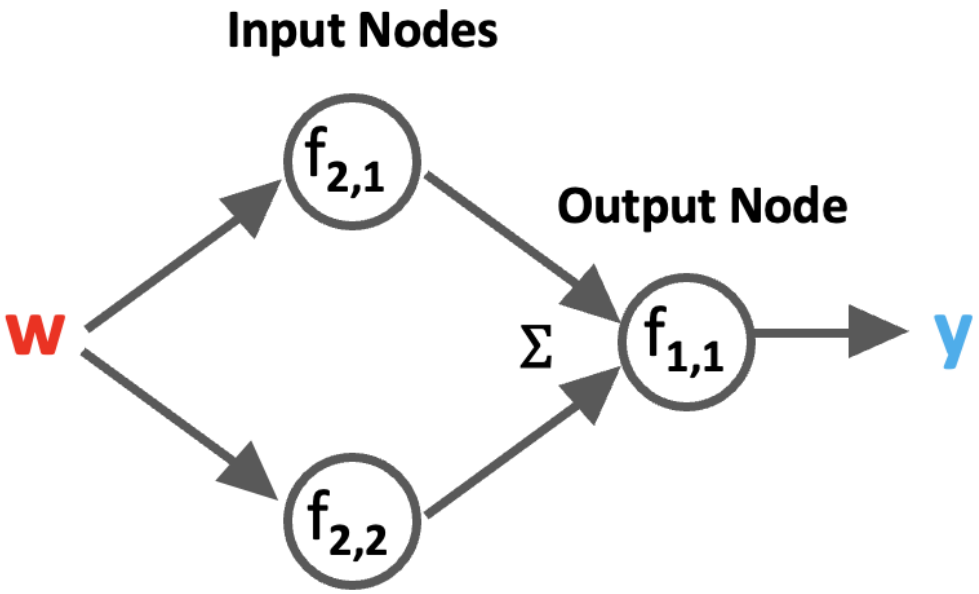

Figure: Adding parallel edges, $y = f_{1,1}(f_{2,1}, f_{2,2})$, where $f_{2,1} = f_{2,1}(w)$, and $f_{2,2} = f_{2,2}(w)$.

- Using Multivariable Chain Rule y is differntiable with respect to any of the variables in the chain, $\frac{\partial y}{\partial w} = \frac{\partial f_{1,1}}{\partial f_{2,1}} \frac{\partial f_{2,1}}{\partial w} + \frac{\partial f_{1,1}}{\partial f_{2,2}} \frac{\partial f_{2,2}}{\partial w}$

Back Propagation

Scaling from a Single Perceptron to a Multilayer Neural Network

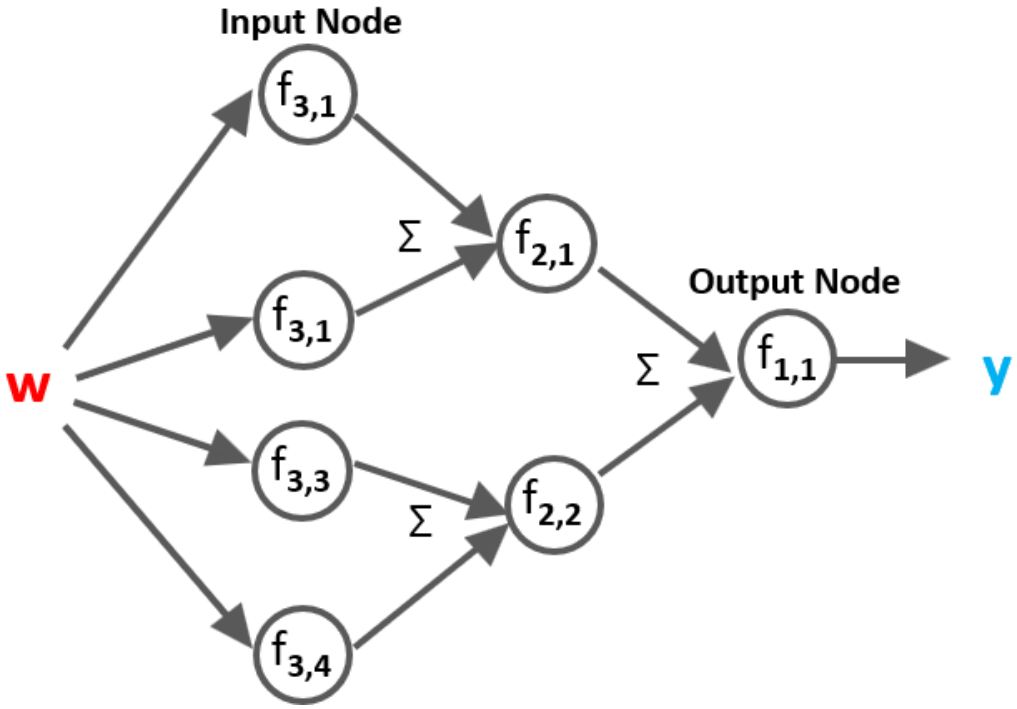

Figure: Adding multiple layers between the input and output layers, where $y = f_{1,1}(f_{2,1}, f_{2,2})$, $f_{2,1}(f_{3,1}, f_{3,2})$, $f_{2,2}(f_{3,3}, f_{3,4})$, $f_{3,1} = f_{3,1}(w)$, $f_{3,2} = f_{3,2}(w)$, $f_{3,3} = f_{3,3}(w)$, and $f_{3,4} = f_{3,4}(w)$.

- The gradient from input to output is $\frac{\partial y}{\partial w} = \frac{\partial f_{1,1}(f_{2,1}, f_{2,2})}{\partial w} = \frac{\partial f_{1,1}(f_{2,1}(f_{3,1}, f_{3,2}),\ f_{2,2}(f_{3,3}, f_{3,4}))}{\partial w}$.

Back Propagation

Scaling from a Single Perceptron to a Multilayer Neural Network

- Using the chain rule expanding on the changes from input to output is $\frac{\partial y}{\partial w} = \frac{\partial f_{1,1}}{\partial f_{2,1}(f_{3,1},f_{3,2})} \frac{\partial f_{2,1}(f_{3,1},f_{3,2})}{\partial w} + \frac{\partial f_{1,1}}{\partial f_{2,2}(f_{3,3},f_{3,4})} \frac{\partial f_{2,2}(f_{3,3},f_{3,4})}{\partial w}$.
- $\frac{\partial y}{\partial w} = \frac{\partial f_{1,1}}{\partial f_{2,1}} \frac{\partial f_{2,1}}{\partial f_{3,1}} \frac{\partial f_{3,1}}{\partial w} + \frac{\partial f_{1,1}}{\partial f_{2,1}} \frac{\partial f_{2,1}}{\partial f_{3,2}} \frac{\partial f_{3,2}}{\partial w} + \frac{\partial f_{1,1}}{\partial f_{2,2}} \frac{\partial f_{2,2}}{\partial f_{3,3}} \frac{\partial f_{3,3}}{\partial w} + \frac{\partial f_{1,1}}{\partial f_{2,2}} \frac{\partial f_{2,2}}{\partial f_{3,4}} \frac{\partial f_{3,4}}{\partial w}$.
- $\frac{\partial y}{\partial w} = \sum_{j=1}^{2} \frac{\partial f_{1,1}}{\partial f_{2,1}} \frac{\partial f_{2,1}}{\partial f_{3,j}} \frac{\partial f_{3,j}}{\partial w} + \sum_{j=3}^{4} \frac{\partial f_{1,1}}{\partial f_{2,2}} \frac{\partial f_{2,2}}{\partial f_{3,j}} \frac{\partial f_{3,j}}{\partial w} = \sum_{j=1}^{4} \prod_{i=1}^{3} \frac{\partial f_{i,j}}{\partial f_{i+1,j}}$ with certain $f_{i,j}$ are excluded, e.g. $f_{2,3}$ is not defined, and $f_{4,j} = w$.
- A better way to express $\frac{\partial f_{1,1}}{\partial w}$ is to define the set of paths that exist from the input node to the output node as $\mathcal{P}$, then $\frac{\partial f_{1,1}}{\partial w} = \sum_{\{j|f_{.,j}\epsilon\mathcal{P}\}}^{n} \prod_{\{i|f_{i,.}\epsilon\mathcal{P}\}}^{m-1} \frac{\partial f_{i,j}}{\partial f_{i+1,j}} \frac{\partial f_{m,j}}{\partial w}$.
- m is the number of layers, and n is the number of nodes from the highest number of nodes layer in the network.

Back Propagation

Scaling from a Single Perceptron to a Multilayer Neural Network

- The number of paths is exponential in any neural network with given layers and nodes. For example, if there are two nodes per layer, and there are ten layers, the number of paths that exist is 2^{10}, but because it's a directed network and using Dynamic Programming the number of computations is significantly reduced.

Back Propagation

Creating Efficiencies in Training Deep Learning

- The idea behind the objective function defined over the entire data set is, e.g., taking an example of MLE $J(\mathbf{w}) = \arg\max_{\mathbf{w}} \sum_{i=1}^{N} log\ P_{model}[\mathbf{x}^{(i)}, y^{(i)} |\ \mathbf{w}]$, that the maximization is over each data point in the data set.
- The question arises, does each data point contribute equally? For example, MSE assumes that each data point contributes equally, because $\frac{1}{N}$
- Furthermore, does there exist a redundancy in data points i.e., does each data point contribute?
- Maximizing the total sum is maximizing the expectation over the data distribution $\hat{P}_{data}$, $J(\mathbf{w}) = \arg\max_{\mathbf{w}} E_{\mathbf{x},y\sim\hat{P}_{data}} log\ P_{model}[\mathbf{x}, y |\ \mathbf{w}]$.

Back Propagation

Efficiencies in Training Deep Learning

- Building further on the argument, the standard error in the sample mean is $\frac{\sigma}{\sqrt{n}}$.
- Where it can be easily seen that the number of samples is not linear, 'n', rather a smaller number of samples contribute $\sqrt{n}$.
- Hence the minibatch technique is used, which takes smaller groups or batches of data points to train the model.
- The sizes of batches have bearing on the underlying hardware architecture. It also creates other types of errors such as could increase variance.
- The online method which takes one sample or batch at a time results in the stochastic gradient descent.